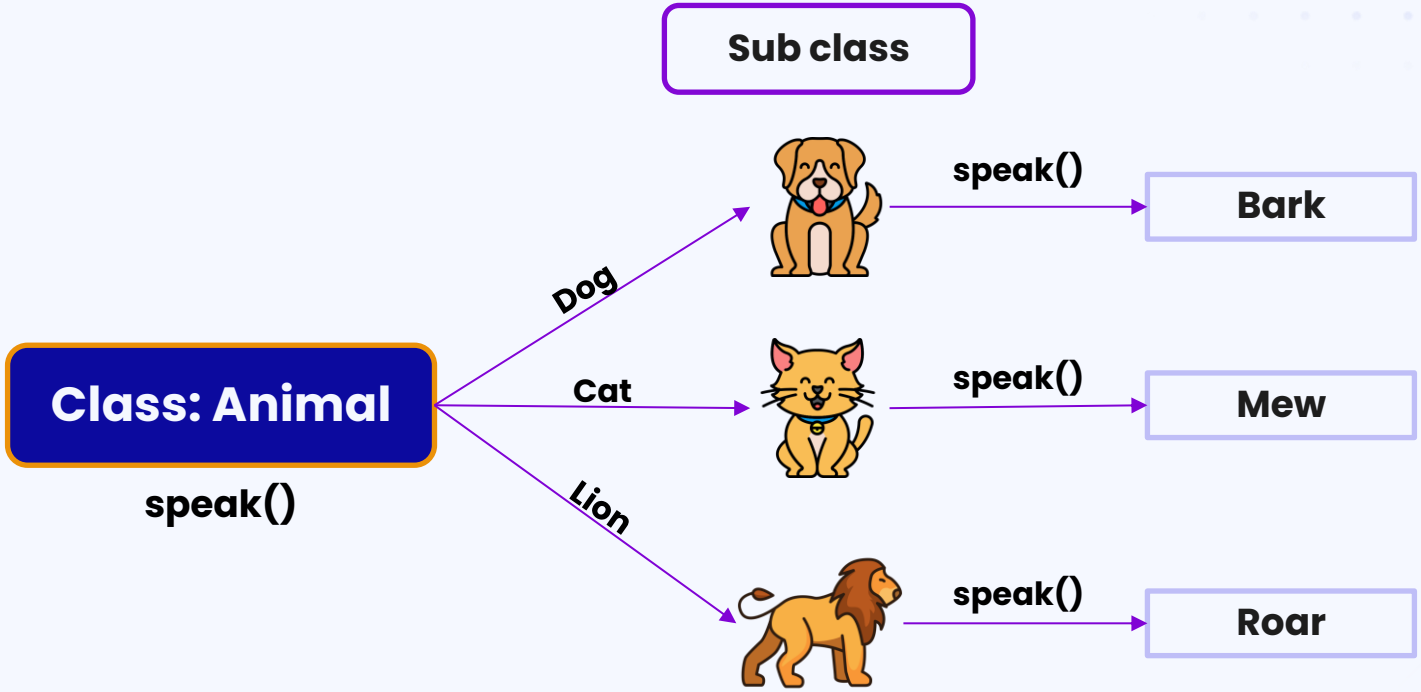
Polymorphism

Polymorphism - Meaning

- **Polymorphism** means "**one thing, many forms**."
- In programming, it means **the same function/method/operator name can work with different types of data/objects**, and Python will do the "right" thing based on what you pass.

In simple words:
Same function, **different** behavior → polymorphism.

Polymorphism Example



Why Polymorphism?

- **Same** code, **multiple** object types
- Cleaner code (less **`if-else`**)
- **Plug-and-play** design
- Focus on **behavior** (Pythonic duck typing)
- Easy **testing** & **maintenance**

Main Areas

In Python, **polymorphism** appears in **4 broad ways**:

- Built-in functions
- Operator Overloading
- Method Overloading
- Method Overriding (Runtime Polymorphism)
- Duck Typing

Polymorphism in Python: Main Types

A) Function Polymorphism (Built-in functions)

Python functions like **`len()`**, **`max()`**, **`print()`** can behave differently for different inputs.

Python	Output
`print(len("Hello"))` `print(len([1, 2, 3, 4]))`	5 4

Why this works:

- **Strings** have a “length”
- **Lists** also have a “length”
- Python uses each object’s **internal behavior**.

Polymorphism in Python: Main Types

B) Operator Polymorphism (Operator Overloading)

Same operator behaves differently with different types.

Python	Output

`print(10 + 20)`
`print('Hi' + 'Amar')`
`print(([1, 2] + [3, 4]))`
	`30 (addition)`
	`Hi Amar (string concatenation)`
	`[1, 2, 3, 4] (list concatenation)`

So **+** has multiple **"forms"** based on operands.

Important note:
This is possible because Python classes define special methods like:

- **__add__** for **+**
- **__mul__** for *****
- **__len__** for **len()**

Polymorphism in Python: Main Types

C) Method Polymorphism (Same method name in different classes)

Different classes can have the same method name, but different implementation.

Example: speak() in different classes

Python	Output

`class Dog:`
`  def speak(self):`
`  return "Bark!"`
`class Cat:`
`  def speak(self):`
`  return "Meow!"`
`animals = [Dog(), Cat()]`
 	
`for a in animals:`
`  print(a.speak())`
	Bark!
	
	Meow!

Polymorphism in Python: Main Types

D) Method Overriding (Same method name in different classes)

In OOP, a child class can override a method of the parent class.

Python	Output

`class Animal:`
`  def speak(self):`
`  return "Some sound"`
 	
`class Dog(Animal):`
`  def speak(self):`
`  return "Bark!"`
 	
`class Cat(Animal):`
`  def speak(self):`
`  return "Meow!"`
	`print(Animal().speak())` → Some sound
	
	`print(Dog().speak())` → Bark!
	
	`print(Cat().speak())` → Meow!

Duck Typing

- Python uses a concept called **Duck Typing**:
- **“If it looks like a duck and quacks like a duck, it’s a duck.”**
- **Meaning:** Python **doesn’t care** about the class name/type strictly.
- It **cares** whether the object **supports** the required behavior.

Duck Typing

Python	Output

`class Bike:`
`  def start(self):`
`  return "Bike started!"`
 	
`class Car:`
`  def start(self):`
`  return "Car started!"`
 	
`def begin(vehicle):`
`  return vehicle.start()`
 	
`print(begin(Bike()))`
`print(begin(Car()))`
	`Bike started!`
	
	`Car started!`

- No inheritance needed. Just same method name **`start()`** and it works.
- This is one of the **most common** real-world **polymorphism** styles in Python.

WATCH

Level up your coding with each episode in this focused Python series.

Next Video!

Dataclass